



PropMetrik

Product & Technical Audit Handoff

Valuations

Property Management

Deals / CRM

Projects

Platform & Admin

Portals & RBAC

A structured reference for an external engineering team engaged to **audit the product, remediate defects, close feature gaps, and verify readiness** for corporate-client onboarding and SOC certification. It lists each service's features and capabilities, the purpose of every sub-tab, all authenticated portals and the RBAC roles they must be tested under, and the gaps identified per service.

Scope included: Valuations · Property Management (+ Tenant Portal) · Deals/CRM · Projects · Platform/Admin · Integrations · Communications · Authentication & RBAC · all end-user portals.

Explicitly excluded: Analytics/market-intelligence dashboards · Blockchain / crypto payout (multi-chain) features.

Classification: Confidential — for the engaged audit team only. · **Architecture:** Next.js (App Router) frontend · Express/TypeScript backend · Python valuation engine · PostgreSQL · Keycloak identity.

Every claim in this document is grounded in a direct read of the codebase on branch `main`; file paths are cited so findings are independently verifiable.

1 Executive Summary

PropMetrik is a multi-tenant proptech SaaS for the Ghanaian market composed of four separately-subscribed customer services on one platform, plus a platform/admin layer and several external portals.

1.1 What the product is

Service	Purpose	Primary users
Valuations	RICS-compliant multi-method property valuation, reconciliation & sealed report production.	Valuers, reviewers/QA, finance officers, instructing clients (as records).
Property Management	Landlord/facilities operations: properties & multi-unit buildings, applications, tenancies/leases, rent & utility billing, maintenance, inspections, portfolio financials.	Property managers, leasing/maintenance staff, accounts officers, tenants (self-service portal).
Deals / CRM	Real-estate sales CRM: leads/contacts, deal pipeline, agent territories, commissions, marketing automation, e-sign.	Sales agents, sales managers, admins; buyers/sellers via public marketplace.
Projects	Construction & development project management: phases/milestones, schedule/Gantt, RFIs, submittals, change orders, procurement, draws, financials.	Project managers, QS, site supervisors; vendors via tokenized bid portal.

These are unified by a **shared platform layer**: authentication/RBAC, an org-wide **Integrations Hub** (BYO OAuth/api-key/webhook), a shared **Communications Hub** (chat · mail · social · calendar), a centralized notification engine, fiat payments (Paystack), and a staff **Admin** console.

1.2 Overall readiness posture (for the auditor's attention)

Headline risks to prioritise. An internal audit rated the codebase ~4/10 overall. The highest-severity, cross-cutting items an external auditor should validate first:

- **CRITICAL** — **Dev-mode auth bypass** in the backend falls back to the first `super_admin` on multiple error paths when dev flags are set. Must be provably disabled in production config (single highest-risk control).
- **CRITICAL** — **Unauthenticated Python valuation engine** (CORS *+credentials on `0.0.0.0:8001`) is called **directly from the browser**, exposing the engine and its DB-backed comparable search.
- **HIGH** — **Fail-open RBAC on the frontend** ("no scoping defined = allow") with hardcoded fallbacks; every route must be re-enforced server-side. Prioritise `viewer` (never writes), `service_admin` (only inviter), and finance/commission (money) tabs across all services.
- **HIGH** — **Money-out has no disbursement rail**: commission payouts, refunds and deposit releases are tracked/approved but never actually paid; money-in (Paystack) works. Money paths need dedicated test coverage.
- **HIGH** — **Two non-production portals present**: the Contractor portal is an unauthenticated mock stub, and there is no external client/owner or valuation-client portal. These must be built & tested or removed before a corporate/SOC handoff.
- **MEDIUM** — **API-URL contract violations** (~24+ files hardcode `http://localhost:4000` instead of the relative `/api` proxy), a dev/prod-leak risk; and single-prod-DB with local-dev-hits-prod operational risk.

Full, per-service detail and file citations follow. Section 8 consolidates the security & SOC-readiness backlog.

1.3 How to read this document

Status tags used throughout: **BUILT** production-wired **PARTIAL** exists but incomplete/UI-only **GAP** not built / stub / must build+test

Section 2 documents authentication, the full RBAC role model, and the complete portal inventory (the answer to "which portals must be tested, and which need building"). Sections 3–6 document each service (purpose, sub-tab table, capabilities, roles, gaps). Section 7 covers the platform/admin layer. Section 8 is the consolidated security/SOC gap backlog. Section 9 is the portal-and-role test matrix the external team should execute for readiness.

Contents

- 1. Executive Summary
- 2. Authentication, RBAC & Portals
 - Auth & session architecture · Staff-vs-customer model · Role matrix (all services) · Portals inventory · Role-scoped sub-portals · Portal/RBAC gaps
- 3. Valuations Service
 - Sub-tabs · Valuation methods · Reports & e-sign · Roles & client access · Gaps
- 4. Property Management Service (+ Tenant Portal)
 - Sub-tabs · Tenant portal · Roles · Gaps
- 5. Deals / CRM Service
 - Sub-tabs · Capabilities · Roles & customer touchpoints · Gaps (incl. phantom-table remediation)
- 6. Projects (Project Management) Service
 - Sub-tabs · Capabilities · Roles & external portals · Gaps
- 7. Platform, Admin, Integrations & Communications
- 8. Consolidated Security & SOC-Readiness Backlog
- 9. Portal & Role Test Matrix (readiness checklist)

Analytics dashboards and blockchain/crypto payout features are intentionally out of scope and are omitted or noted only where they intersect a shared surface.

2 Authentication, RBAC & Portals

Source of truth: code on branch `main`. Written for an external SOC audit handoff; every claim cites a file path.

2.1 Auth & session architecture

Identity providers & token model

Authentication is split between a Next.js session layer (`frontend/src/auth.ts` , NextAuth v5) and an Express verification layer (`backend/src/middleware/auth.ts`). Four frontend providers:

Provider id	Purpose	Backend call	Resulting <code>userType</code>
<code>credentials</code>	Email/password staff & customer login	<code>POST /api/v1/auth/login</code>	from backend (staff/customer)
<code>tenant-credentials</code>	Tenant portal login	<code>/tenant-portal/auth/keycloak/password-login</code>	<code>tenant</code>
<code>keycloak</code>	Enterprise SSO (OIDC)	Keycloak issuer	from token realm roles
<code>google</code>	Social sign-in/up	<code>POST /api/v1/auth/google</code> (verifies Google <code>id_token</code> server-side)	from backend

- Session strategy is **JWT**, 30-day max age. The NextAuth JWT stores `accessToken` (backend-issued local JWT or a Keycloak access token), `role` , `tier` , `userType` , `subscribedServices` , `organizationId` . Token refresh is built in (Keycloak `refresh_token`; credentials via `/auth/refresh`); failure sets `session.error='RefreshTokenError'` .
- The backend `authenticate` middleware accepts three credential types in order: **(1) Keycloak JWT** verified against remote JWKS (issuer+audience checks; Redis blacklist for logout/revocation), **(2) Local JWT** via `config.jwt.secret` , **(3) Dev-mode bypass** loading the first `super_admin` when `AUTH_DEV_BYPASS=true` and non-prod. Tokens may also arrive as a `?token=` query param for SSE/EventSource. After auth, `enrichUserFromDb` populates `userType` / `tier` and **fails closed to `customer`** (never staff) — correct least-privilege default.
- Analytics API keys (`pmk_`) are explicitly rejected on session routes.

AUDIT FLAG — dev bypass. Multiple `catch` branches in `authenticate` fall back to the dev `super_admin` user on *any* auth failure when `config.app.devAuthBypass / env==='development'` is set. This must be provably disabled in production via a config assertion + test. It is the single highest-risk control for SOC.

Subdomain model & host-only cookie isolation

Three hosts: `propmetrik.com` (main app), `tenant.propmetrik.com` (tenant portal), `api.propmetrik.com` (backend); `app.*` is legacy/dead. Host routing lives in `frontend/src/middleware.ts` (`NEXT_PUBLIC_TENANT_HOST` -driven, nothing hardcoded). On the tenant host the portal **is the whole site** (staff routes never served). NextAuth runs `trustHost:true` and derives the auth URL from the request host, so session cookies are **host-scoped, not domain-scoped** — a tenant session on `tenant.*` is never sent to `propmetrik.com` and vice-versa. **This host-only cookie isolation is the primary tenant/staff separation boundary and should be a named SOC control.**

Staff vs Customer model & backend authorization layers

- `organizations.is_platform_org=true` marks the single PropMetrik staff org → members are `user_type='staff'`; everyone else `'customer'` .
- Admin console requires role AND `user_type='staff'`** (`requirePlatformRoles`): a customer holding an `admin` role still fails because `isPlatformStaff` is false. Frontend mirrors this (Admin tab dropped for customers). Staff bypass subscription tiers; tiers apply to customers only.
- Backend layers guards: `requirePMAccess` / `requirePMWrite` (roles) → **project-scoped membership** (`getProjectMembership`) with 8 per-member permission flags. Org-leadership roles bypass project scoping; non-admins only see projects they own/are active members of — enforced on path params, allow-listed child-resource params (injection-safe), and query-param list endpoints.

2.2 RBAC role model

Platform / staff roles (org-internal realm roles)

From `frontend/src/lib/rbac.ts` (lower level = more authority). Live map is fetched from `GET /api/v1/rbac/config` (5-min cache) with these as fallbacks.

Role	Lvl	Access scope
<code>super_admin</code>	1	Full platform; bypasses all RBAC/tier/service/project checks everywhere
<code>firm_principal</code>	10	Director/Principal Valuer; full service access (all services + construction/budget/data_hub)
<code>admin</code>	15	Org admin; full services; Admin console only if <code>user_type='staff'</code>
<code>senior_valuer</code>	20	Lead valuer / QA; valuations
<code>manager</code>	25	Team manager; broad access across all services
<code>project_manager</code>	28	projects, analytics, property_management, construction, budget
<code>valuer</code>	30	Valuations only
<code>agent</code>	30	CRM/Deals only
<code>finance_manager</code>	35	Finance/billing sub-tabs (valuations finance)
<code>compliance_officer</code>	40	Valuations only

probationer / inspector	50/55	Trainee (supervised) / field inspections (+ PM read)
analyst	60	Read-only analytics + valuations
viewer	90	Read-only basic
tenant	—	Tenant portal only (/dashboard/tenant); never sees staff tabs

Customer service roles (per subscribed service)

Customer roles are scoped per service (`user_service_subscriptions.service_role`). `service_admin` = full access within a service and the only role that can invite teammates (`requireCanInvite`). Type = customer for all rows.

Service	Role	Access scope
Valuations	service_admin	Full: valuations, finance, clients, calendar, analytics, team, settings
	senior_associate	valuations, clients, calendar, analytics, team
	associate	valuations, clients, calendar, team
	finance_officer	valuations, finance
	viewer	valuations (read)
CRM / Deals	service_admin	Full incl. pipelines, integrations, team
	sales_manager	deals, agents, financials, commissions, analytics, workflows, campaigns
	sales_agent	deals, properties, contacts, tasks, docs, messaging, calendar, targets
	viewer	deals/properties/contacts (read)
Projects	service_admin	Full incl. settings, team
	project_manager	overview, construction, procurement, financials, docs, units, analytics
	site_supervisor	overview, construction, docs, comms
	quantity_surveyor	overview, procurement, financials, docs
	viewer	overview, docs, units (read)
Property Mgmt	service_admin	Full incl. team, integrations
	property_manager	properties, tenants, maintenance, portfolios, vendors, applications
	leasing_agent	properties, tenants, applications, docs, calendar
	maintenance_coordinator	maintenance, vendors, tenants, docs
	accounts_officer	financials, portfolios, tenants, docs
	viewer	overview/properties/tenants (read)

Backend enforcement: `requireServiceAccess(serviceKey)` gates the service (staff/leadership bypass, else active subscription); `requireServiceRole(serviceKey, [...])` gates sensitive writes to specific roles. Internal roles bypass subscription checks for their authorized services only (`ROLE_SERVICE_ACCESS`).

2.3 Portals inventory (what must be tested; what needs building)

Every distinct authenticated (or public-but-must-test) experience, confirmed against code.

Portal	Who logs in	URL / Route	Auth method	Status	Notes
Staff/platform dashboard	PropMetrik staff	propmetrik.com/dashboard/*	NextAuth credentials/keycloak/google	BUILT	Full nav incl. Admin tab
Customer org dashboard	Corporate customer	propmetrik.com/dashboard/*	Same; nav filtered by subscriptions + role	BUILT	No Admin tab; tabs gated by subscribed services + <code>service_role</code>
Tenant portal	Tenant (<code>user_type='tenant'</code>)	tenant.propmetrik.com/dashboard/tenant/*	tenant-credentials → Keycloak; host-only cookie isolation	BUILT	Payments, maintenance, documents, lease e-sign, messages, profile
Tenant apply/onboarding (public)	Prospective tenant (unauth, magic-link)	/tenant/apply/[id] , /tenant/set-password , /tenant/application/[id]/status , /tenant/lease/[id]	Magic link / OTP / Keycloak set-password	BUILT	Separate /app/tenant/* tree — duplication flag (two tenant trees)
Public marketplace +	Anyone (unauthenticated)	propmetrik.com/marketplace	None (public)	BUILT	search, property-by-token,

enquiry					inquiries, application-link (mounted no-auth)
E-Sign external signer	Any invited signer (email link)	/sign/[token]	Token-based (optionalAuth)	BUILT	Draw/type/upload signature, decline. Token is the credential
Developer / API console	Customer org w/ API product	propmetrik.com/developers/*	NextAuth session + requireApiProductAccess	BUILT	Self-serve pmk_ keys, usage, plan, streaming
Vendor bid portal	Invited vendor/contractor	/vendor/bid/[token]	Token-based, no login	BUILT	View bid, submit line-items/attachments, Q&A, decline — fully wired
Staff Admin console	Staff admin / super_admin only	/dashboard/admin/*	requireAdmin = role AND user_type='staff'	BUILT	Customer cannot reach even with admin role
Contractor portal	Contractor	/contractor-portal	None wired	GAP / STUB	Renders mock data only ; no auth, no API, not linked in any nav. Build or remove before audit
Client/owner project portal	Project owner/client (external)	—	—	GAP	No external surface to view draws, approve/sign change orders, or answer RFIs
Valuation instructing-client view	Instructing/valuation client	—	—	GAP	No share-token report route; clients are records only . Report viewing is staff-only
Buyer/seller CRM portal	Marketplace buyer/seller	—	—	GAP	Only public marketplace + enquiry; no logged-in deal-tracking portal for the customer side

2.4 Role-scoped in-app "sub-portals" (test independently per service)

- **Deals/CRM:** agent / sales_agent (own pipeline, no commissions/pipelines/team) vs sales_manager (financials, commissions) vs service_admin (full). Commissions default admin-only.
- **Valuations:** valuer / associate (author) vs senior_valuer / senior_associate (QA/review) vs finance_manager / finance_officer (finance tab only) vs compliance_officer. No external instructing-client view (gap above).
- **Projects:** project_manager (owns projects) vs org-admin (all) vs per-member permission flags (can_view_financials etc.). External contractor = stub; vendor = token bid portal.
- **Property Management:** staff/customer PM roles vs the **tenant** portal (separate host, separate session) — the clearest staff-vs-external split and strongest isolation story.

2.5 Portal & RBAC gaps (SOC portal-readiness)

Portal gaps

- **HIGH Contractor portal is a non-functional stub** — serves hardcoded mock data, no auth, not linked. Build real auth or remove; an unauthenticated route that *looks* like a portal is itself a finding.
- **MEDIUM No valuation instructing-client / external report portal**, and **no buyer/seller CRM portal**, and **no project client/owner portal**. If any are promised capabilities, they are unbuilt.
- **MEDIUM Two parallel tenant trees** (/app/tenant/* public onboarding vs /app/dashboard/tenant/* authed). Confirm which is canonical and that the legacy tree exposes no authed data unauthenticated.

Auth hardening to verify for SOC

- Prove AUTH_DEV_BYPASS + dev-user fallback are disabled in prod (config assertion + test).
- **Token-in-query-string** (?token=) for SSE — verify stripped from access logs / not cached by the proxy.
- **Public/no-auth route inventory** for pen-test scope: marketplace (search, inquiries, property-by-token), /vendor/bid/:token , /sign/:token , public PM invoice pay/verify endpoints, tenant magic-link/OTP/set-password, OAuth callback. Confirm token entropy, single-use, expiry for each.

- **Local-JWT fallback** trusts `config.jwt.secret` — confirm rotation and that local vs Keycloak JWTs cannot be confused (audience checks).

RBAC coverage to test independently

- Frontend RBAC is a fetched config with hardcoded fallbacks — test the fallback path (backend `/rbac/config` unreachable) does not over-grant. `canAccessServiceSubTab / canCustomerAccessSubTab` **fail open** ("no scoping defined = allow"); confirm the backend re-enforces every such route.
- Frontend button-hiding for integration scoping is still TODO — server-side enforced, but UI may show actions a role cannot perform.
- Test the **staff-AND-userType** admin gate with a customer holding an `admin` role (the exact bypass the gate closes) — a named test case.
- Prioritise, per service: `viewer` (must never write), `service_admin` (only inviter), and finance/commission (money) tabs.

Key files for the auditor: `frontend/src/middleware.ts`, `frontend/src/auth.ts`, `frontend/src/lib/rbac.ts`,
`backend/src/middleware/{auth,serviceAccess,pmAuth}.ts`, `backend/src/routes/{tenantPortal,developerPortal,marketplace,bid-management,eSign}.ts`, `backend/src/index.ts`.

3 Valuations Service

RICS-compliant property valuation workspace: valuers run a subject property through a guided multi-method wizard (up to 8 RICS methods computed by a Python engine), reconcile the methods into a single opinion of value with sensitivity analysis, and produce a sealed, e-signed, branded valuation report — with client, invoicing, team and document-vault management around it.

3.1 Sub-tabs

Nav in `frontend/src/app/dashboard/valuations/layout.tsx` ; role-gating in `rbac.ts` (`FALLBACK_valuationTabAccess`).

Sub-tab	Route	Purpose / Use case	Key features
Valuations (list)	<code>/dashboard/valuations</code>	Home list of all valuation jobs	List + filters; entry into per-valuation workflow
New Valuation	<code>/valuations/new</code>	Intake wizard for a new job	Subject-property form; auto-creates a DRAFT + autosaves; <code>?client_id</code> prefill; inline "Generate with AI" writeups
Per-valuation workflow	<code>/valuations/[id]/...</code>	The valuation itself, as an ordered wizard	Property Setup → Floor Plans → HBU → Method Selection → method pages → Reconciliation → Report; step order method-driven (<code>valuation-workflow.ts</code>); sub-routes: property, floor-plan, hbu, methods, comparables, market, cost, rental-market, income, drc, profits, residual, reconciliation, report, documents, subject
Team	<code>/valuations/team</code>	Manage the valuation team	Invites, roles, Keycloak wiring
Finance	<code>/valuations/finance</code>	Invoicing & fees	Ghana Valuation Fee Calculator (GhIS scales), invoices, KPIs
Clients	<code>/valuations/clients</code>	Instructing-client registry	Individual/Corporate/Government; overview/invoices/valuations tabs; "New Valuation" passes <code>?client_id</code>
Calendar	<code>/calendar?service=valuations</code>	Scheduling (shared, service-scoped)	Shared dashboard calendar filtered to valuations
Documents (Vault)	<code>/valuations/documents</code>	Central document vault across all valuations	Version history w/ status badges — Current (approved) / Superseded / Draft; upload/download/delete
Integrations	<code>/valuations/integrations</code>	Org-level BYO connectors	Admin-only; shared Integrations Hub
Settings	<code>/valuations/settings</code>	Org/firm configuration	General, Branding , Fee Defaults, Reports, Approvals (approval chains), API Keys, Notifications

3.2 Valuation methods

All computed by the Python engine (`backend/src/services/valuation-engine/python/app/methods/`), orchestrated by Node routes; the frontend method pages source inputs and render results.

Method	What it values
Sales Comparison	Market value by adjusting a basket of comparable sales to the subject (comparable search → market analysis)
Cost	Replacement/reproduction cost new less depreciation, plus land — for buildings with limited market evidence
DRC (Depreciated Replacement Cost)	Specialised/owner-occupied assets via feature-driven MEA (Method, Economic-life, Age) depreciation
Profits	Trade-related property (hotels, schools, healthcare, fuel stations) from divisible-balance / trading potential
Residual	Development land / sites: GDV less costs, fees, finance and developer's profit
Income	Investment property via pro forma + direct capitalisation + DCF (rental-market evidence → income engine)
Market Rent	Open-market rental value by adjusting rental comparables (feeds Income + standalone rent opinions)
Land Value	Land value by reconciling a residual approach with land comparable sales (GhIS strength-based weighting)
Multi-method reconciliation + sensitivity	Combines applied methods into one reconciled opinion; sensitivity does real per-method re-runs per RICS VPS 3 (<code>POST /sensitivity</code>)

3.3 Reports & e-sign

Three renderers exist: an **HTML viewer**, a **signed DOCX→PDF** pipeline (LibreOffice), and a **PDFKit cover** page; structured Phase-2 report data (cover, certification, legal, risk, reconciliation table, TOC) is assembled server-side. AI narrative sections are **draft-only** — the valuer edits before approval; narratives are grounded strictly in real data (coordinates, comparable rates, Google Places amenities). Firm branding from **Settings → Valuation Company Branding** flows into the DOCX/PDF report and cover and the invoice email (PROPMETRIK fallback); the **HTML report-viewer letterhead is**

NOT branded. On approval the report is **locked-on-seal**: a **SHA-256 tamper hash** is computed (a tamper-evidence hash — **not** a cryptographic/PKI signature), any prior approved version is marked **superseded**, and an optional **client e-sign** can be triggered (external signer signs via the token page `/sign/[token]`). Revisions go reject-for-revision → new version, surfaced in the Document Vault as Current/Superseded/Draft.

3.4 Roles & client access

Internal roles: `firm_principal` (Director/Principal Valuer), `senior_valuer` (Lead/QA/reviewer-approver), `valuer` (independent valuer), `finance_manager`, `compliance_officer`, plus `probationer` / `inspector` / `analyst`. Customer roles: `service_admin`, `senior_associate`, `associate`, `finance_officer`, `viewer`. Approval/QA is internal (chains default to `senior_valuer`). **The instructing client is a data record, not a portal user** — there is **no external instructing-client report view**; the only external touchpoint is the token e-sign page.

3.5 Gaps identified

- **CRITICAL Python engine security posture** — unauthenticated with CORS `*+credentials` on `0.0.0.0:8001`, and the **browser calls it directly** (`NEXT_PUBLIC_PYTHON_API_URL`), exposing the engine and its DB-backed land-comparable search.
- **HIGH Digital seal is a hash, not a signature** — SHA-256 tamper-evidence over the PDF; proves integrity but not signer identity / non-repudiation cryptographically.
- **MEDIUM No external instructing-client report portal** — clients are records only; no authenticated view of their report status/version history.
- **MEDIUM Conflicting method-weight tables** — reconciled value depends on which code path runs; 3–4 different weight tables exist (TS `valuationEngineService.ts`, Python `config.py`, `multi_method.py`) and none reads a single DB source.
- **MEDIUM Income page hardcodes cap-rate / risk-premium / discount-rate constants client-side**, against the DB-config direction.
- **LOW HTML report viewer unbranded**; generated report content is cached in `valuation_reports.content` (template fixes need a manual regenerate); Ghana `DateField` `dd/mm/yyyy` sweep still pending across many pages; reconciliation weight-display NaN% observed; ~4,300 LOC orphaned engine code + stale 5-region taxonomy (defaults unknown regions to Greater Accra vs 16 production regions).

4 Property Management Service (+ Tenant Portal)

PropMetrik's landlord/facilities operations suite — manages a portfolio of properties and multi-unit buildings, rental applications, tenancies/leases, rent & utility billing, maintenance work orders routed to vendors, inspections, documents and portfolio financials. Paired with a self-service **Tenant Portal**. Top-nav in `PMTopNav.tsx` ; grouped sub-navs in `PMSectionNav.tsx` .

4.1 Sub-tabs

Sub-tab	Route	Purpose / Use case	Key features
Overview	<code>/property-management</code>	Portfolio-health dashboard	KPI cards (occupancy, income, overdue, work orders, expiring leases, applications); NOI/cap-rate/cash-flow; receivables aging; quick actions
Properties → Listings	<code>/properties</code>	Asset registry incl. multi-unit buildings	Buildings as parent rows + expandable unit roster (lazy-loaded); search/filters; live FX→GHS normalization; add/edit/delete
Properties → Enquiries	<code>/enquiries</code>	Buyer/lessee leads from marketplace	Lead table (sale/lease/rent badge, offer, status workflow); "View in pipeline" → CRM deal; PM-only upsell
Communications	<code>/communications</code>	Unified Chat/Mail/Social/Calendar hub	Wrapper around shared <code>CommunicationsHub</code>
Portfolios → Portfolio	<code>/portfolios</code>	Portfolio analytics & performance	Value w/ YoY, occupancy, collection rate; composition pies; lease-expiry buckets
Portfolios → Financials	<code>/financials</code>	Portfolio P&L / receivables (real, not a stub)	Period-filtered revenue/expenses/NOI/outstanding; charts; receivables aging; PDF export; Payment Settings
Portfolios → Reports	<code>/reports</code>	Enterprise operational reports	Aged Receivables, Vacancy, Performance (NOI), Tenant Turnover, Maintenance; export
Portfolios → Audit	<code>/audit</code>	Compliance/change audit trail	Filterable activity log; action/resource/user summaries; pagination
Portfolios → Bulk Ops	<code>/bulk-operations</code>	Mass actions across properties	Bulk rent increase; bulk work-order creation; export (CSV/JSON/Excel); import (Tenants/Properties)
Portfolios → Brochure	<code>/portfolios/brochure</code>	Shareable portfolio brochure	5-page printable PDF (cover w/ QR, exec summary, geo distribution, asset schedule)
Applications	<code>/applications</code>	Rental application intake → lease pipeline	Status workflow (draft→...→lease_generated→lease_signed); Start Review/Approve/Reject; Generate Lease (e-sign); convert to tenant/tenancy; shareable public application links
Tenants	<code>/tenants</code>	Tenant directory + tenancy/lease mgmt	Roster + status; detail tabs (Tenancies/Utilities/Payments/Maintenance/Documents); generate/renew/terminate lease; apply utility charges; Send Access Invite (Email) ; multi-step new-tenant wizard
Maintenance → Work Orders	<code>/maintenance</code>	Work-order tracking + vendor dispatch	Stats + table; Assign Vendor modal (vendor + date + slot → notifies vendor & tenant) ; Complete (actual cost); status updates
Maintenance → Inspections	<code>/inspections</code>	Property condition reports	Move-in/out/routine/periodic; room-by-room condition items; complete w/ overall condition
Maintenance → Vendors	<code>/vendors</code>	Service-provider directory	Vendor table (categories, rating, jobs); add/edit/delete; performance metrics
Documents	<code>/documents</code>	Property document vault	Summary cards; tabs All/Legal/Financial/Tenant; e-sign signing-status tracking ; view/download/delete
Team	<code>/team</code>	Service team / role management	<code>ServiceTeamManager</code> (invite, assign customer service roles)
Integrations	<code>/integrations</code>	Org BYO integrations	Shared <code>IntegrationsHub</code>

4.2 Tenant Portal

Runs on `tenant.propmetrik.com` ; primary tree is `/app/dashboard/tenant/*` (wrapped in `PortalShell`). Sidebar: Dashboard, Payments, Maintenance, Documents, Messages, Profile/Settings/Lease.

- **Dashboard** — balance / next-due / open-maintenance / lease-progress cards; recent activity; lease summary.
- **Pay rent** — rent schedule + totals + history; pay via **Mobile Money (MTN/Vodafone/AirtelTigo)**, **Bank**, **Card**, **GhQR** (crypto path excluded from scope); live FX-locked GHS-equivalent + fee breakdown; **Auto-Pay** setup; utility charges with **dispute** flow.
- **Maintenance** — submit request w/ category, priority, **up to 5 photos**, preferred date/time, "permission to enter" consent; view status + per-request detail (work-order status).
- **View & sign lease** — in-portal signature capture + terms agreement → e-sign the lease.

- **Documents** — view/search/download shared docs and **upload** own documents.
- **Messages** — threaded landlord↔tenant chat with read receipts (5s polling).
- **Public flows** — application status tracker (timeline) and public apply (validates a manager-issued application-link token).

Auth: Tenants are `userType: 'tenant'` on the tenant subdomain via NextAuth `tenant-credentials` (Keycloak ROPC). Manager sends an email invite → `/tenant/set-password?token=...` (validates magic-link, sets Keycloak password, auto-signs-in). The Keycloak realm has **no SMTP**, so set/forgot-password is served by the app's own 3-tier mailer, not Keycloak.

4.3 Roles (customer RBAC)

`service_admin` (full + only inviter) · `property_manager` (everything except Financials/Team/Integrations) · `leasing_agent` (leasing-focused; no maintenance/vendors/financials) · `maintenance_coordinator` (maintenance/vendors/tenants; no applications/financials) · `accounts_officer` (billing/finance-focused) · `viewer` (read-only subset). Gating is fail-open on unconfigured sub-tabs and must be re-enforced server-side.

4.4 Gaps identified

- **HIGH API-URL contract violations** — `financials/page.tsx:227`, `tenants/[id]/page.tsx:81`, `documents/page.tsx:48` hardcode `http://localhost:4000` instead of the relative `/api` proxy (dev-leak / prod risk).
- **MEDIUM Tenant Auto-Pay is UI-only** (toggles + toast, no persistence API); some tenant payment history controls (Filter/Export/receipt) are cosmetic; non-redirect payment success path uses a `setTimeout` simulation.
- **MEDIUM Staff "New Work Order" form not fully wired** (Save not connected — the tenant-side request form *is* functional); tenant-list "Send Message" / "Payment History" quick actions are stubs.
- **MEDIUM Public application status tracker uses placeholder step logic** ("mock logic for demonstration"; step highlighting not reliably wired to the real status enum — the timeline list itself is real).
- **LOW** Two coexisting tenant trees (`/app/tenant/*` vs `/app/dashboard/tenant/*`) = duplication risk; Inspections has no photo capture (presigned-MinIO pending; tenant maintenance form *does* support photos); large un-split pages + eager Recharts + `limit:100` list fetches flagged for perf; `leases/[id]` lacks e-sign initiation (driven from Applications/Documents instead).
- *Note:* the "Financials GHÇ0 stub" from older memory refers to the **CRM** Financials page — the **PM** financials page is real and data-backed.

5 Deals / CRM Service

A standalone, separately-subscribed real-estate sales CRM for Ghana: leads/contacts, a configurable deal pipeline, agents and geo-fenced territories, listing inventory, commissions and targets, document generation + e-sign, multi-channel communications, and marketing automation (drip campaigns). A thin orchestration layer over shared platform services, scoped by `organization_id`. UI label: "DEAL MANAGEMENT" (`CrmSidebar.tsx`).

5.1 Sub-tabs

Sub-tab	Route	Purpose / Use case	Key features
Deals	<code>/deals</code>	Pipeline root — work deals through stages	Kanban + table, pipeline selector, FilterBuilder, bulk stage-change/tags/delete/export, slide-in panel
Properties	<code>/deals/properties</code>	CRM listing inventory (native + PM-bridged)	Property cards w/ pipeline-stage progress, stacking-plan toggle, filters
Contacts	<code>/deals/contacts</code>	Leads & clients	Lead status/score/budget, CSV import wizard, merge-dedupe, relationship map
Agents	<code>/deals/agents</code>	Agent roster	License, specializations, active/closed deals, rating; filters
Territories	<code>/deals/territories</code>	Geo-fenced territories + lead auto-routing	Mapbox click-to-draw polygon , PostGIS boundaries, exclusive/priority/color
Companies	<code>/deals/companies</code>	Corporate clients / developers / agencies	Industry/type, deal count/value; bulk delete/export
Tasks	<code>/deals/tasks</code>	Task tracking w/ overdue alerts	Complete-checkbox rows, priority/status filters, stats, new-task dialog
Documents	<code>/deals/documents</code>	Deal document template library	Category, stamp-duty/notarization/witness toggles, versioning, "Seed Ghana Templates"
Financials	<code>/deals/financials</code>	Revenue, forecasting, commission payouts, billing	Real 6-tab module (Overview/Commissions/Forecast/Billing/Payments/Settings), KPIs, top-earners, Paystack refunds
Communications	<code>/deals/communications</code>	Unified comms hub	<code>CommunicationsHub</code> → WhatsApp lead chat · Mail · Social · Calendar
Workflows	<code>/deals/workflows</code>	Automation engine (triggers/templates)	Campaign list, trigger-type icons, active/executions/success-rate stats
Pipelines	<code>/deals/pipelines</code>	Pipeline & stage configuration	Stage reorder/edit/delete, deal-type (sale/rental/development), PipelineDesigner
Commissions	<code>/deals/commissions</code>	Commission ledger + payout approvals	Maker-checker payout approvals (records/statements/approvals), calculator, generate-statements, bulk approve
Targets	<code>/deals/targets</code>	Sales targets + leaderboard + achievements	Circular-progress target cards, pacing, leaderboard, badges
Drip Campaigns	<code>/deals/drip-campaigns</code>	Automated email sequences	A/B step variants w/ weights + open/click tracking , audience segments w/ match-preview, delivery log
Team / Integrations	<code>/deals/team</code> , <code>/deals/integrations</code>	Service-team RBAC · BYO connections	<code>ServiceTeamManager</code> · <code>IntegrationsHub</code>
Deal detail / New deal	<code>/deals/[id]</code> , <code>/deals/new</code>	Unified deal workspace · create deal	Clickable stage bar, inline-edit KPIs, doc panel + checklist + signed indicator, unified timeline; new-deal financials grid w/ auto-calc probability

5.2 Capabilities (done vs pending)

Done / live: configurable multi-pipeline deals (real tables); contacts/leads w/ score + CSV import/export + merge-dedupe; **agent territory management** (PostGIS boundaries, click-to-draw, point-in-polygon, auto-route unassigned contact to owning territory's agent — fully real); campaign segmentation + open/click tracking; drip engine w/ weighted A/B step variants; **commissions maker-checker** (records→statements→approvals, plans/tiers/splits/clawback); targets/leaderboard; Paystack refunds; real 6-tab Financials; document templates + e-sign; Communications hub; FX normalization + enterprise RBAC dispatcher on money endpoints.

Pending / partial: offers/reservations/earnest holds (no dedicated schema — modeled as pipeline stages); purchase installment plans (metadata only); **outbound payout rail** (commissions tracked but **never actually disbursed**); lead-scoring engine (field + property-match only, no AI); social syndication (connection UI exists, no publishing service); voice calling, KYC, i18n.

5.3 Roles & customer touchpoints

Staff roles: `agent` (deals/contacts/tasks/targets/comms), plus `finance_manager` (financials/commissions) and `manager / admin`. Customer roles: `service_admin`, `sales_manager`, `sales_agent`, `viewer`. **Customer-facing touchpoints are public only:** the marketplace (listings) + enquiry. The enquiry endpoint is **capture-first, deal-gated**: PM listings always capture into `property_inquiries`; a CRM deal is auto-created only if the org has the CRM service (via `ensureCrmMirror()`). **Gap:** there is **no logged-in buyer/seller portal** — post-enquiry, the buyer cannot track their deal.

5.4 Gaps identified

Phantom CRM-table bug — remediated in source (status update). The systemic defect (code querying non-existent `crm_deals / crm_contacts / crm_tasks /...` when the real tables are un-prefixed `deals / contacts / tasks /...`; `tsc` -green but runtime-broken) is now fixed in all originally-broken source files (0 phantom refs in `backend/src`). **Residual risk:** phantom references survive in 12 non-source artifacts — the stale compiled `dist/` (will re-break on redeploy if not rebuilt), integration tests, `scripts/check-crm-tables.mjs`, and one migration. **No build-time SQL-schema guard exists, so this class of bug can silently regress** — the recommended generated-types/repository layer is still a gap.

- **HIGH No outbound disbursement rail** — commission payouts, refunds and deposit releases are tracked/approved but never actually paid (money-in works; money-out absent). Highest-risk money-movement gap.
- **MEDIUM** No dedicated offers/reservations/installments schema; no AI lead-scoring engine; social syndication publishing pipeline absent; no buyer/seller customer portal.
- **LOW** Merge-field registry historically shipped empty (confirm if template merge is depended on); voice calling, KYC (Ghana Card captured but not verified), i18n, Sentry (integrated but `SENTRY_DSN` unset), backup/DR codification outstanding.

6 Projects (Project Management) Service

A Procore-style construction & development command center for Ghanaian developers — from land acquisition through permits, construction, procurement, financials and handover. Subscription-gated. Two-tier nav: 8 group tabs (Overview, Construction, Procurement, Financials, Documents, Communications, Units, Settings) each expanding to sub-items; inside a project, a per-project sub-nav (Milestones, Change Orders, RFIs, Submittals, Photos, Punch Lists, Site Logs, Procurement, Budget/Cost, Draws, Issues/Risks).

6.1 Sub-tabs / sections (selected)

Section	Route	Purpose / Use case	Key features
Projects list	/projects	Portfolio hub of all developments	Grid/list, filters, project cards (progress, budget), portfolio KPIs
Project detail	/projects/[id]	Single-project command center	Tabbed Overview/Schedule/Budget/Phases/RFIs/Submittals/Change Orders/Punch Lists/Documents; Gantt; budget line items; team + contractor assignments
Schedule (Gantt)	/projects/gantt , /[id]/schedule	Cross-project & per-project timeline	Phase bars, actual-progress fill, milestone markers, critical path, drag-to-reschedule autosave
Milestones	/[id]/milestones	Delivery checkpoints incl. Ghana statutory	List/Timeline, create/start/complete, sub-phases
RFIs	/projects/rfis , /[id]/rfis	Request-for-Information lifecycle	Create/submit/close, ball-in-court + "Awaiting Me" tabs , cost/schedule impact, overdue stats
Change Orders	/projects/change-orders	Change management with e-sign	Cost+schedule impacts, submit for approval, Request E-signature (branded PDF) , approve/execute, ball-in-court
Submittals	/projects/submittals	Shop drawings / product data review	By type+spec section, revision tracking, review workflow, ball-in-court
Draws / Pay Apps	/[id]/draws	Payment draw requests	Line items by cost code, retainage/net, submit→approve→fund, e-signature request , record funding
Procurement (Contracts)	/projects/procurement	Purchase orders	PO w/ line items, Draft→...→Delivered workflow, PDF, approve/reject
Bid Management	/projects/bids	Competitive bidding + tokenized vendor portal	Publish + invite vendors by email, submission analytics, comparison table, Q&A, award
Contractors	/projects/contractors	Contractor registry	Trade, bank + mobile-money, license/TIN, ratings, project counts
Budget / Costs	/projects/financials , /[id]/budget-cost , /costs	Revenue/collection + per-project budget	Invoiced/collected/outstanding KPIs, budget-vs-actual variance, at-risk projects
Invoices / Estimator	/projects/invoice-builder , /cost-estimator	Invoicing & parametric estimation	Visual builder + approval workflow; GFA/region/spec estimate → "Convert to Project"
Issues / Risks · Punch Lists · Safety · Site Logs · Timesheets · Equipment · Drawings · Meetings · Closeout · Checklists	/projects/...	Full field-ops suite	Kanban issues + risk register; deficiency tracking; incident/OSHA; daily diary; GPS timesheets; equipment; drawing register w/ revisions; minutes; warranties; QA templates
Communications · Units · Integrations · Reports · Settings · Audit Log	/projects/...	Comms, unit sales, connectors, config	Shared hubs; unit stats; Xero/QuickBooks/Procore/DocuSign catalog; SOC-2 activity log; E-Signature settings manager

6.2 Capabilities (verified in code)

- **Auto-seeded Ghana phase + milestone plan** — type-specific templates (single-house 6, multi-residential 8, commercial 7) with Ghana-statutory milestones (development/building permit, EPA/EIA, fire cert, habitation/occupancy cert, handover); EPA-permit logic (>40 units / >5000 sqm) codified.
- **Gantt schedule** (drag-to-reschedule, critical path); **kanban issues** + risk register.
- **RFI/submittal/change-order ball-in-court routing** — `ball_in COURT` derived in SQL, "Awaiting Me" filter, "awaiting your response" notifications.
- **Change Orders / Draws / Contractor Contracts e-sign** — config-driven Pattern B: 3 per-org config tables, each opt-in (`trigger_on_approval` / `auto_trigger_on_assignment`, min-amount threshold, signer roles, auto-execute/auto-fund). The manual "Request E-Signature" button always works; auto-trigger requires the org to enable the type.
- **Invoice payments** — idempotent `confirmPayment` (webhook + manual), unique- `reference` ledger upsert, Paystack (subaccount split + fee engine + FX→GHS) + NOWPayments.
- **Project-scoped access + 8 permission flags** — org-leadership sees all; others restricted to owned/active-member projects; flags `can_view`, `can_edit`, `can_approve_costs`, `can_upload_documents`, `can_add_tasks`, `can_manage_team`, `can_view_financials`, `can_receive_notifications`; guards on path, child-resource, list-query and financials params.

- **Tokenized vendor bid portal** — the one real external portal: emailed magic-link lets invited vendors view a bid, submit, decline, ask Q&A, download attachments via real public API.

6.3 Roles & external portals (built vs gap)

Internal roles: staff `project_manager` (+ leadership) with read-level `finance_manager / inspector / analyst / viewer`; customer roles `service_admin`, `project_manager`, `site_supervisor`, `quantity_surveyor`, `viewer` (sub-tab-scoped). These are all internal (subscribed-org) roles.

External portals — BUILT: the **Vendor bid portal** (`/vendor/bid/[token]`) is wired end-to-end. (The Tenant portal exists but belongs to Property Management.)

External portals — GAP:

- **GAP Contractor portal** (`/contractor-portal`) — UI shell only: renders from a hardcoded `mockData` object, no API, no auth, not linked in any nav. "Submit Invoice / Update Progress / Upload Photos" are inert. Non-functional stub.
- **GAP Client / owner portal** — does not exist. No external route for an owner/client to view draws, approve/sign change orders, or answer RFIs. E-sign `signer_roles` reference `owner_representative / lender_representative / contractor_representative` but there is no portal surface for them.

6.4 Gaps identified

- **HIGH** Contractor external portal is a mock stub (build+wire+test or remove); no client/owner external portal (biggest construction-workflow handoff gap).
- **MEDIUM** PM e-sign auto-trigger is opt-in and off by default (only manual "Request E-Signature" works out of the box); **API-URL contract violations in ~24 files** (hardcoded `http://localhost:4000`); `[id]/team/page.tsx:15` hardcodes `organizationId='' // TODO`, breaking team scoping on that route.
- **LOW** `/bids` vs `/bidding` overlap (likely redundant); duplicated `[id]/X` vs top-level `X` pages (~3,300 LOC — checklists/punch-lists/site-logs/procurement); no `next/dynamic` (eager Recharts), fetch waterfalls + per-keystroke refetch + unbounded list fetches (perf against remote prod DB); inspections photo capture (presigned-MinIO) pending.

7 Platform, Admin, Integrations & Communications

The shared layers underneath the four services. (Analytics dashboards and blockchain/crypto payout are out of scope.)

7.1 Admin service (F8 — platform-staff console)

The PropMetrik-operator control plane at `/dashboard/admin/*`. **Access gating:** `admin/layout.tsx` is a **server component** that calls `auth()` and redirects *before* any admin markup/JS is sent; access requires **both** a platform role (`super_admin / admin` — deliberately not `firm_principal`, the org-owner role every subscriber gets) **and** `userType==='staff'`. (This server-component gate is the remediation of an earlier client-side-only redirect finding; the auditor should verify backend admin routes enforce the same role+user_type check.)

Sub-tab	Purpose / use case
Overview	Operator console: stat cards (users, orgs, active 24h, properties, valuations MTD, security events), live system-status (API/PostgreSQL/Redis/object-storage), recent activity.
User management	Platform-wide user directory; destructive hard-delete user (<code>DELETE /admin/users/:id</code> , email-confirm; removes account + Keycloak login, irreversible).
Organizations	Org directory; Downgrade to free (cancel paid sub, keep data) and hard-delete org (name-confirm; removes org + all member accounts platform+Keycloak + subscriptions/invoices/payments).
Platform billing / revenue	The Platform Revenue dashboard: period selector; Total Revenue, Subscriptions, Platform Fees, MRR/ARR, Active Subs, Processed Volume, Pending/Overdue; Transactions + Invoices ledgers. Principle: platform revenue = subscriptions + platform fees only (customer principal excluded).
RBAC config	RBAC Policy Manager: bind <code>resource_type + action</code> → <code>allowed_roles</code> (14 roles) + scoping predicates (<code>require_ownership / require_assignment / require_same_org</code>); invalidate RBAC cache; Audit tab of allow/deny decisions.
Integrations (platform-owned)	Catalog of ~35 platform-level services (Payments, Auth, Messaging, Infra, AI/ML, Maps, Data, Documents, Verification) with health overlay. Where Paystack and Identity/KYC (Ditid / Ghana Card — not yet configured) live — distinct from the org BYO hub.

7.2 Integrations Hub (org-wide BYO framework)

One UI behind every service's Integrations tab (`IntegrationsHub.tsx`), driven by the backend catalog (`integrationCatalog.ts`). Each provider is tagged **model** (`platform / byo_org / byo_user / hybrid`) + **auth** (`oauth2 / api_key / webhook / managed`). A BYO-OAuth provider is only "Connect"-able once the platform OAuth app creds exist in env (else "Requires setup"). Org-shared integrations are gated to leadership+finance server-side (403); `byo_user` creds are scoped to org AND `user_id` (no cross-teammate inbox leak).

Provider	Type	Model	Status	Notes
Xero	oauth2	byo_org	LIVE	Only accounting integration live end-to-end (syncs project costs as bills)
Gmail · Google Calendar	oauth2	byo_user	LIVE	Inbox sync + log to records · two-way calendar sync
Custom Webhook	webhook	byo_org	LIVE	Events to a URL you control; optional signing secret
WhatsApp Business	managed	platform	LIVE	Info-only; templates/reminders run by PropMetrik
QuickBooks · Outlook · Google Drive · OneDrive	oauth2	byo_org/user	AVAILABLE	Wired; need app creds (Outlook/OneDrive live-confirmed in history)
Facebook · Instagram · TikTok · LinkedIn · X	oauth2	byo_org	AVAILABLE	Publish listings; TikTok live-confirmed. Shared publish pipeline (approved photos)
Twilio SMS	api_key	byo_org	AVAILABLE	Auth Token AES-256-GCM encrypted at rest ; requires <code>CREDENTIALS_ENCRYPTION_SECRET</code>
Zapier · Connect Website	api_key	byo_org	AVAILABLE	5000+ app bridge · listings on own site + enquiries back
Twilio Voice	api_key	hybrid	COMING SOON	Click-to-call; not connectable

Encrypted secret store. `backend/src/utils/secretCrypto.ts` — reversible AES-256-GCM (PBKDF2 key from `CREDENTIALS_ENCRYPTION_SECRET`) for outbound BYO credentials that must be read back (e.g. an org's Twilio Auth Token), distinct from the one-way `api_key_hash`. **SOC note:** this reversible key is a single-point root of trust for every org's outbound credential — confidentiality + rotation of `CREDENTIALS_ENCRYPTION_SECRET` is a named control.

7.3 Communications Hub (shared PM / Deals / Projects)

One tabbed shell (`CommunicationsHub.tsx`) — **Chat · Mail · Social · Calendar** — adapting to the `service` prop. **Chat:** WhatsApp lead chat (Deals) or landlord↔tenant messaging (PM). **Mail:** synced Gmail/Outlook inbox, per-user scoped + optional per-record filter, bodies rendered in a **sandboxed iframe with no allow-scripts** (email JS cannot execute — good hardening); reply/forward + "Link to record". **Compose:** docked Gmail-style window (chips, rich text, attachments) carrying `entity_type / entity_id` so sent mail attaches to a record. **Social:** WhatsApp inbox + TikTok/FB/IG publishers. **Calendar:** shared view filtered by service. (Internal team chat stays in the separate floating Workspace messenger.)

7.4 Notifications & Payments

Notifications — one canonical `notify()` core (`backend/shared-services/notifications/notify.ts`) fans an event across in-app (staff/tenant tables), email (3-tier failover: MS Graph → SES → SMTP), SMS (Twilio incl. BYO), and SSE realtime; routed by `audience` (staff vs tenant) + `category` ; **never throws** (channel failures collected, business action never broken); cron reminders (rent, CRM tasks, drip). *Debt*: a legacy parallel PM notification service still exists and should be folded in.

Payments (fiat) — Paystack (cards + Mobile Money) with sub-accounts so managers receive payouts directly and the platform takes a split fee; central `paymentProcessor.ts` across deal/project/valuation/subscription rails + ledger + webhook reconcile; daily subscription renewal + dunning; idempotent invoice `confirmPayment` ; FX-settled USD-rent-in-GHS at a live locked rate (dual-currency ledger). **This is a live money path with material audit findings** (see §8.3): verify race + non-transactional success path, non-atomic FIFO, five Paystack clients / three HMAC impls, opt-in webhook signature verification, float money math.

8 Consolidated Security & SOC-Readiness Backlog

Synthesised from the internal `docs/audit/` (19 reports; overall 4/10, Security 3/10 — the lowest dimension) plus the per-service findings above. The auditor changed no code and cited `file:line` throughout. Most severe issues are **systemic** — one bad pattern at a shared chokepoint — so fixes are high-leverage. This is the recommended external-auditor priority order.

Positives to preserve (audit-confirmed): zero committed secrets (correct `.gitignore`; only `.example envs`); clean `notify()` core; clean Workspace org-isolation; host-only cookie tenant/staff isolation; region-partitioned `properties` table; sandboxed no-script email rendering; fail-closed-to-customer default in `enrichUserFromDb`.

#	Severity	Area	What to fix / verify
1	CRITICAL	Authentication bypass paths	<code>POST /auth/google</code> reportedly trusts client-supplied <code>{email,googleId}</code> with no token verification (mints a JWT for any email) — <i>note the frontend now verifies the <code>id_token</code> server-side; confirm the backend route does too</i> ; dev-auth-bypass fail-open (verify the <code>config</code> hardening closes it across rate-limit/authenticate/scan paths); <code>JWT_SECRET</code> constant fallback if env unset; <code>AppError</code> prototype bug makes <code>instanceof UnauthorizedError/ForbiddenError</code> always false (auth-error branches never fire). Any one enables takeover.
2	CRITICAL	E-sign is not real cryptography	Signatures hash the PDF URL string, not the bytes ; private keys "encrypted" with XOR under a hardcoded dev secret ; TSA timestamps are a self-signed in-memory mock — on a live valuation + lease signing path. Non-repudiation is currently fake. (Valuation "digital seal" is likewise a SHA-256 hash, not a PKI signature.)
3	CRITICAL	Money-path integrity	Non-transactional / raceable payment confirmation (double-record or lost rent); non-atomic FIFO application; org-unscooped commission mutations; five Paystack clients / three webhook-HMAC impls (some sign <code>JSON.stringify(body)</code> not raw bytes); opt-in webhook signature verification (returns "valid" when the secret is unset); float money math. No outbound disbursement rail exists (payouts tracked, never paid).
4	HIGH	Cross-org IDOR (systemic)	Unscoped role/member mutations (<code>team.ts</code> , <code>serviceTeam.ts</code> — a <code>service_admin</code> of org A could alter users in any org); <code>rbac.ts</code> admin gate missing <code>user_type='staff'</code> (a customer with an <code>admin</code> role could rewrite platform-wide policies); e-sign envelope status leaks any envelope to any caller; world-open unauthenticated Python valuation engine (CORS *, browser calls it directly).
5	HIGH	Server-side RBAC enforcement	Frontend RBAC fails open ("no scoping defined = allow") with hardcoded fallbacks; confirm every route is re-enforced server-side and org-scoped BYO 403s are consistent (button-hiding is not the boundary). Prioritise <code>viewer / service_admin / finance</code> tabs per service, and the staff-AND-userType admin gate.
6	HIGH	Single prod DB + local-dev-hits-prod	One production PostgreSQL; local dev (<code>NODE_ENV=development</code>) connects to production data (no dev/staging DB). Amplifies every fail-open auth path against prod and is the root of the performance profile (remote latency, pool max 10).
7	MEDIUM	Secrets rotation & blast radius	<code>CREDENTIALS_ENCRYPTION_SECRET</code> (reversible AES key for all BYO outbound creds) and <code>ESIGN_KEY_SECRET</code> are single-point roots of trust — define rotation / Vault / HSM posture. (Positive: nothing committed.)
8	MEDIUM	Retention, soft-delete & audit trail	Irreversible hard-deletes (DB + Keycloak) with no retention window; two competing audit stacks; token blacklist never written (logout doesn't revoke tokens); no defined data-retention policy.
9	MEDIUM	Frontend controls & test coverage	No CSP/HSTS on the Next origin; <code>eslint.ignoreDuringBuilds:true</code> ; 47 backend tests but 0 frontend tests , no coverage gate, no frontend CI — no automated guard on regressions, including money paths. ~24+ files hardcode <code>http://localhost:4000</code> (dev/prod-leak).

Legacy-debt items (parallel notification/e-sign/Paystack stacks, dead-code trees) are documented in the audit but are quality/maintainability rather than SOC-blocking.

9 Portal & Role Test Matrix (readiness checklist)

The distinct authenticated experiences the external team must exercise for corporate-onboarding + SOC readiness, and the roles each must be tested under. "Build" = must be built before it can be tested.

9.1 Portals to test / build

Portal	State	Roles / identities to test under	Priority test focus
Staff/platform dashboard	TEST	super_admin, admin, manager, firm_principal, valuer, agent, project_manager, finance_manager, viewer	Tab visibility per role; that non-privileged roles cannot reach money/admin surfaces
Customer org dashboard	TEST	Per service: service_admin, the mid roles, and viewer (x Valuations/CRM/PM/Projects)	Subscription gating; viewer never writes; only service_admin can invite; finance/commission tabs
Tenant portal	TEST	tenant (on tenant.* host)	Host-only cookie isolation (tenant session cannot reach staff app); pay/maintenance/lease-sign; cross-tenant IDOR
Tenant apply/onboarding (public)	TEST	Unauthenticated + magic-link token	Token entropy/single-use/expiry; set-password flow; no authed-data leak from legacy /app/tenant/* tree
Public marketplace + enquiry	TEST	Unauthenticated	Enquiry capture; deal-gating (deal only if org has CRM); no injection via public search/inquiry
E-sign external signer	TEST	Token identity (email link)	Envelope-status IDOR (any envelope to any caller — a known finding); signature integrity (see §8.2)
Vendor bid portal	TEST	Invited vendor (token)	Token scope to one bid; submit/decline/Q&A; cannot view other vendors' bids
Developer / API console	TEST	Customer org w/ API product; staff	API-key issuance/rotation; entitlement gate; rate-limit; pmk_ keys rejected on session routes
Staff Admin console	TEST	staff_admin / super_admin — and negative: customer holding an admin role	role AND user_type='staff' gate; backend admin routes enforce it; hard-delete confirmations
Contractor portal	BUILD	(none — needs real auth first)	Currently mock+unauth+orphaned. Build real auth or remove before audit
Project client/owner portal	BUILD	owner_representative, lender_representative	External RFI answers, change-order approval/sign, draw visibility — does not exist
Valuation instructing-client view	BUILD	instructing client	No external report view/status; clients are records only — build if promised
Buyer/seller CRM portal	BUILD	marketplace buyer/seller	No logged-in deal-tracking portal — build if promised

9.2 Cross-cutting RBAC test cases (all services)

- **Fail-open fallback:** make backend /rbac/config unreachable → confirm the frontend hardcoded fallback does not over-grant, and every route is still enforced server-side.
- **Cross-org IDOR sweep:** as service_admin of org A, attempt to read/mutate org B's members, roles, deals, tenants, projects, envelopes, commissions.
- **Least-privilege writes:** every viewer role attempts a write on each sub-tab (must 403); every non-finance role attempts finance/commission/payout actions.
- **Invite control:** only service_admin can invite/assign roles (requireCanInvite); non-admins blocked server-side.
- **Admin gate:** customer user with an admin role cannot reach /dashboard/admin nor any backend admin route (role AND user_type='staff').
- **Money paths:** replay/duplicate webhooks; unsigned webhook with secret unset (must reject, not accept); concurrent payment confirm (no double-record / lost rent).
- **Session isolation:** a tenant.* session cookie must never authenticate against propmetrik.com and vice-versa.

Suggested remediation sequence for corporate + SOC readiness:

1. Close the CRITICAL auth-bypass paths (§8.1) and prove dev-bypass is off in prod.
2. Replace the e-sign crypto with a real PKI/TSA provider (§8.2) — it underpins legal valuation + lease documents.
3. Make the money path transactional + verify webhook signatures + scope commission mutations by org (§8.3); then build the outbound disbursement rail.
4. Sweep cross-org IDOR + authenticate the Python engine + re-enforce fail-open RBAC server-side (§8.4–8.5).
5. Stand up a dev/staging DB to end local-dev-hits-prod (§8.6).
6. Decide each GAP portal (§9.1): build+test or remove; remove the unauthenticated contractor stub either way.
7. Add CSP/HSTS, a frontend test suite + CI coverage gate, secrets-rotation policy, and a data-retention/soft-delete policy (§8.7–8.9).

— End of handoff. All findings are grounded in a direct read of branch main; the internal audit reports live under docs/audit/ for deeper evidence.